CS 423
Operating System Design: Semaphores
10/15

Ram Alagappan

Slide ack: Prof. Shivaram Venkataraman (Wisconsin)

Logistics

Today - Semaphores

10/20 - Midterm review

10/22 - Midterm exam

Midterm: in-class, all material up to 10/15, one side A4 cheat sheet allowed

Most questions will be reasoning and problem solving

RECAP

Concurrency Objectives

Mutual exclusion (A and B don't both run critical section)

- solved with *locks*

Ordering (B runs only after A has done its thing)

- solved with *condition variables* and *semaphores*

Producer/Consumer: Two CVs and WHILE

```
void *producer(void *arg) {
    for (int i = 0; i < loops; i++) {
        Mutex_lock(&m); // p1
        while (numfull == max) // p2
            Cond_wait(&empty, &m); // p3
        do_fill(i);  // p4
        Cond_signal(&fill); // p5
        Mutex_unlock(&m); //p6
    }
}
```

```
void *consumer(void *arg) {
    while (1) {
        Mutex_lock(&m);
        while (numfull == 0)
            Cond_wait(&fill, &m);
        int tmp = do_get();
        Cond_signal(&empty);
        Mutex_unlock(&m);
    }
}
```

SUMMARY: 3 RULES OF THUMB FOR CVS

1. Need to keep some state in addition to CV's

2. Always wait or signal with lock held

3. Always recheck state when thread wakes up from a wait

   Use a while instead of an if

HOARE VS MESA SEMANTICS

Mesa: used widely (cheaper & easier to implement)

- Signaler holds on to lock & the CPU/core
- Signal moves a waiting thread to READY/RUNNABLE state
- Not necessary that the signaled waiter runs next

Conditions may change by the time waiter runs

Hoare: almost never used (expensive & complex to implement)

- Signal gives lock & the CPU/core to waiting thread
- Waiter runs when woken up by the signaler
- When waiter finishes, it hands lock + CPU/core back to signaler

Conditions remain unchanged because waiter runs immediately after

END RECAP

Semaphores

Condition variables have no state (other than waiting queue)

- Programmer must track additional state (e.g., done variable in the join example)

Semaphores have state: track integer value

State embedded with the primitive itself

- State cannot be directly accessed by user program, but state determines behavior of semaphore operations

Equivalence

Semaphores are equally powerful to Locks+CVs
- what does this mean?

One might be more convenient, but that's not relevant

Equivalence means each can be built from the other



SEMAPHORE OPERATIONS

Allocate and Initialize

```
sem_t sem;
sem_init(sem_t *s, int initval) {
  s->value = initval;
}
```

User cannot read or write value directly after initialization

SEMAPHORE OPERATIONS

Wait or Test: sem_wait(sem_t*)

Decrements sem value by 1, Waits if value of sem is negative (< 0)

Signal or Post: sem_post(sem_t*)

Increment sem value by 1, then wake a single waiter if exists

Wait and post are atomic

BINARY Semaphore (LOCK)

```
typedef struct __lock_t {
    sem_t sem;
} lock_t;

void init(lock_t *lock) {

}

void acquire(lock_t *lock) {

}

void release(lock_t *lock) {

}
```

sem_init(sem_t*, int initial)
sem_wait(sem_t*): Decrement, wait if value < 0
sem_post(sem_t*): Increment value
then wake a single waiter

Chat for 2 mins…



thread_join using Semaphores

```
void thread_join() {
    mutex_lock(&m);       // w
    while (done == 0)     // x
      cond_wait(&c, &m); // y
    mutex_unlock(&m);     // z
}
```

```
void thread_exit() {
    mutex_lock(&m);       // a
    done = 1;             // b
    cond_signal(&c);      // c
    mutex_unlock(&m);     // d
}
```

---

sem_t s;
sem_init(&s, _____);

sem_wait(): decrement, wait if value < 0
sem_post(): increment, wake a single waiter

```
void thread_join() {
        sem_wait(&s);
}
```

```
void thread_exit() {
        sem_post(&s)
}
```

Thread_join with semaphores

If it is any +ve number, join will _____.

If it’s any -ve number, join will _____.

Producer/Consumer: ATTEMPT#1

Single producer thread, single consumer thread
Single shared buffer between producer and consumer

Use 2 semaphores

- sem_init(&emptyBuffer, __);
- sem_init(&fullBuffer, __);

```
Producer
while (1) {
    sem_wait(&emptyBuffer);
    produce(&buffer);
    sem_post(&fullBuffer);
}
```

```
Consumer
while (1) {
    sem_wait(&fullBuffer);
    consume(&buffer);
    sem_post(&emptyBuffer);
}
```

One slot but many P/C

Assume still one buffer

But **many** producers and consumers

Will this code behave correctly?

- sem_init(&emptyBuffer, 1);
- sem_init(&fullBuffer, 0);

```
Producer
while (1) {
    sem_wait(&emptyBuffer);
    produce(&buffer);
    sem_post(&fullBuffer);
}
```

```
Consumer
while (1) {
    sem_wait(&fullBuffer);
    consume(&buffer);
    sem_post(&emptyBuffer);
}
```

Producer/Consumer: ATTEMPT#2

Again back to single producer thread, single consumer thread

Shared (circular) buffer with **MAX** elements between producer and consumer

Use 2 semaphores

- sem_init(&emptyBuffer, __);
- sem_init(&fullBuffer, __);

Producer

```
void put(int value) {
    buffer[fill] = value;
    fill = (fill + 1) % MAX;
}

while (1) {
    sem_wait(&emptyBuffer);
    put(value)
    sem_post(&fullBuffer);
}
```

Consumer

```
int get() {
    int tmp = buffer[use];
    use = (use + 1) % MAX;
    return tmp;
}

While (1) {
    sem_wait(&fullBuffer);
    consume();
    sem_post(&emptyBuffer);
}
```

Producer/Consumer: ATTEMPT#3

Final case:

- Multiple producer threads, multiple consumer threads
- Shared buffer with N elements between producer and consumer

Requirements

- Each C thread must consume from a unique filled element
- Each P thread must produce into a unique empty element

P/C Multiple Threads

Producer

```
void put(int value) {
    buffer[fill] = value;
    fill = (fill + 1) % MAX;
}

while (1) {
    sem_wait(&emptyBuffer);
    put(value)
    sem_post(&fullBuffer);
}
```

Consumer

```
int get() {
    int tmp = buffer[use];
    use = (use + 1) % MAX;
    return tmp;
}

While (1) {
    sem_wait(&fullBuffer);
    consume();
    sem_post(&emptyBuffer);
}
```

Would this work correctly with multiple P/C threads?

What do we need here for correctness?

P/C Multiple Threads: ATTEMPT#1

Consider 3 possible locations for mutual exclusion

```
Producer
    sem_wait(&mutex);
    sem_wait(&emptyBuffer);
    put(value)
    sem_post(&fullBuffer);
    sem_post(&mutex);
```

```
Consumer
    sem_wait(&mutex);
    sem_wait(&fullBuffer);
    consume();
    sem_post(&emptyBuffer);
    sem_post(&mutex);
```

QUESTION: Is it correct?

P/C Multiple Threads: ATTEMPT#2

```
Producer
    sem_wait(&emptyBuffer);
    sem_wait(&mutex);
    put(value)
    sem_post(&mutex);
    sem_post(&fullBuffer);
```

```
Consumer
    sem_wait(&fullBuffer);
    sem_wait(&mutex);
    consume();
    sem_post(&mutex);
    sem_post(&emptyBuffer);
```

QUESTION: Is it correct?

P/C Multiple Threads: ATTEMPT#2

```
Producer
    sem_wait(&emptyBuffer);
    sem_wait(&mutex);
    put(value)
    sem_post(&mutex);
    sem_post(&fullBuffer);
```

```
Consumer
    sem_wait(&fullBuffer);
    sem_wait(&mutex);
    consume();
    sem_post(&mutex);
    sem_post(&emptyBuffer);
```

Question: Can you make this more concurrent?

P/C Multiple Threads: ATTEMPT#2

```
Producer
    sem_wait(&emptyBuffer);
    sem_wait(&mutex);
    put(value)
    sem_post(&mutex);
    sem_post(&fullBuffer);
```

```
Consumer
    sem_wait(&fullBuffer);
    sem_wait(&mutex);
    consume();
    sem_post(&mutex);
    sem_post(&emptyBuffer);
```

Question: Can you make this **even more** concurrent?

Hint: think what happens if payloads are large…

General Rule

What should we initialize a semaphore to?

Set it to the #resources being handed out

- Lock 1: there's only 1 resource being acquired
- CV 0: there's nothing to acquire
- Producer/Consumer - set to N: #buffers to fill or consume

Reader/Writer Locks

Let multiple reader threads grab lock (shared)

Only one writer thread can grab lock (exclusive)

- No reader threads
- No other writer threads

Let us see if we can understand code…

Reader/Writer Locks

```
1 typedef struct _rwlock_t {
2     sem_t lock;
3     sem_t writelock;
4     int readers;
5 } rwlock_t;
6
7 void rwlock_init(rwlock_t *rw) {
8     rw->readers = 0;
9     sem_init(&rw->lock, 1);
10    sem_init(&rw->writelock, 1);
11 }
```

Reader/Writer Locks

```
13 void rwlock_acquire_readlock(rwlock_t *rw) {
14       sem_wait(&rw->lock);
15       rw->readers++;
16       if (rw->readers == 1)
17           sem_wait(&rw->writelock);
18       sem_post(&rw->lock);
19 }
21 void rwlock_release_readlock(rwlock_t *rw) {
22       sem_wait(&rw->lock);
23       rw->readers--;
24       if (rw->readers == 0)
25           sem_post(&rw->writelock);
26       sem_post(&rw->lock);
27 }
29 rwlock_acquire_writelock(rwlock_t *rw) { sem_wait(&rw->writelock); }
31 rwlock_release_writelock(rwlock_t *rw) { sem_post(&rw->writelock); }
```

T1: acquire_readlock()
T2: acquire_readlock()
T3: acquire_writelock()
T2: release_readlock()
T1: release_readlock()
// who runs?
T4: acquire_readlock()
// what happens?
T5: acquire_readlock()
// where blocked?
T3: release_writelock()
// what happens next?

Reader/Writer Locks

```
13 void rwlock_acquire_readlock(rwlock_t *rw) {
14       sem_wait(&rw->lock);
15       rw->readers++;
16       if (rw->readers == 1)
17           sem_wait(&rw->writelock);
18       sem_post(&rw->lock);
19 }
21 void rwlock_release_readlock(rwlock_t *rw) {
22       sem_wait(&rw->lock);
23       rw->readers--;
24       if (rw->readers == 0)
25           sem_post(&rw->writelock);
26       sem_post(&rw->lock);
27 }
29 rwlock_acquire_writelock(rwlock_t *rw) { sem_wait(&rw->writelock); }
31 rwlock_release_writelock(rwlock_t *rw) { sem_post(&rw->writelock); }
```

T1: acquire_readlock()
T2: acquire_readlock()
T3: acquire_writelock()
T4: release_readlock()
// what happens?
// what's the problem?

Build Semaphore using Lock and CV!

```
Typedef struct {
    int value;
    cond_t cond;
    lock_t lock;
} zem_t;

void zem_init(zem_t *s, int value) {
    s->value = value;
    cond_init(&s->cond);
    lock_init(&s->lock);
}
```



zem_wait(): Waits while value <= 0, Decrement
zem_post(): Increment value, then wake a single waiter

Build Zemaphore from LOCKs AND CV

```
zem_wait(zem_t *s) {
    lock_acquire(&s->lock);
    while (s->value <= 0)
        cond_wait(&s->cond);
    s->value--;
    lock_release(&s->lock);
}
```

```
zem_post(zem_t *s) {
    lock_acquire(&s->lock);
    s->value++;
    cond_signal(&s->cond);
    lock_release(&s->lock);
}
```

zem_wait(): Waits while value <= 0, Decrement
zem_post(): Increment value, then wake a single waiter



Semaphores

Semaphores are equivalent to locks + condition variables

- Can be used for both mutual exclusion and ordering

Semaphores contain **state**

- How they are initialized depends on how they will be used
- Init to 0: Join (1 thread must arrive first, then other)
- Init to N: Number of available resources

Sem_wait(): Decrement and then wait if < 0 (atomic)

Sem_post(): Increment value, then wake a single waiter (atomic)

Can use semaphores in producer/consumer and for reader/writer locks

CONCURRENCY SUMMARY SO FAR

Motivation: Parallel programming patterns, multi-core machines

Abstractions, Mechanisms

- Spin Locks, Ticket locks
- Queue locks
- Condition variables
- Semaphores

NEXT STEPS

Next class: Midterm Review